

6.3 Elements of Dynamic Programming

Dynamic programming refers to the process of recursively breaking a problem into subproblems, then combining those solutions to create a solution for the main problem. We've seen some examples of using dynamic programming, but when exactly should we choose this technique? We will discuss two key properties about a problem that can suggest that dynamic programming is a viable way to solve the problem.

6.3.1 Optimal Substructure

The first property is called **optimal substructure**. A problem exhibits optimal substructure when the optimal solution contains within it optimal solutions to subproblems. This was true for both the weighted scheduling problem and the rod cutting problem.

The following is a loose guide on how you might go about discovering that a problem has this property.

1. **Making a choice.** At some point in the algorithm there is a choice to make. Making the choice leaves one or more subproblems to be solved.
2. **Decomposable problem.** If the correct choice is made, the optimal solution can be described by combining the solutions from the induced subproblems.
3. **The subproblem solutions are optimal.** We must also be careful to check that using the solutions to subproblems directly are optimal.

Question 71. State the choices made during the weighted scheduling problem, what the subproblems are, and why the solutions to them are optimal.

For input n , we want the best schedule using events $1, \dots, n$. ①

1. We either keep or reject the last event. ②

2. Subproblems after making a choice ③
 $\rightarrow \text{schedule}(p(n)) \quad \rightarrow \text{schedule}(n-1)$

3. If someone we trust told us the correct choice, we only need to worry about the new subproblems.

Question 72. State the choices made during the rod cutting problem, what the subproblems are, and why the solutions to them are optimal.

For rod w/ length n , what is the best way to cut it to maximize profits? ①

1. Where do we make the leftmost cut? ②



2. Subproblems: BestCut for the right half of the rod? ③

3. Prove that it's optimal...

Not every problem has this property!! Be careful that you are not misguided into using dynamic programming when it isn't applicable. Let's look at two similar looking problems to see this in action.

Problem (SHORTESTPATH).

INPUT: An unweighted graph $G = (V, E)$ and the labels u, v of two vertices in G .

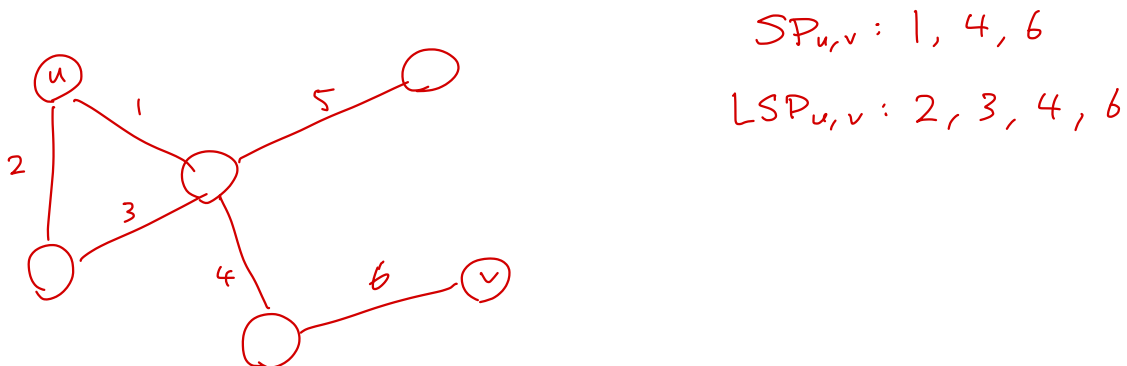
OUTPUT: A path from u to v consisting of the fewest edges.

Problem (LONGESTSIMPLEPATH).

INPUT: An unweighted graph $G = (V, E)$ and the labels u, v of two vertices in G .

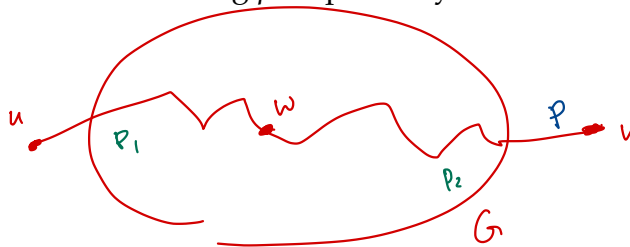
OUTPUT: A path from u to v consisting of as many edges in G as possible. The path must be simple, meaning each ~~edge~~ vertex can only be used once.

Question 73. Draw example inputs and outputs for SHORTESTPATH and LONGESTSIMPLEPATH, where the graph G has 6 vertices. Use the same graph for both examples.



Let's first look at the SHORTESTPATH problem. Again, to identify optimal substructure, we want to make sure that the optimal solution is just a combination of optimal solutions to smaller problems.

Suppose that we have the optimal path p from u to v in our graph. To check for optimal substructure, we decompose our solution into a few pieces. In this case let's split it into two pieces by choosing a vertex w that is on this path. Let p_1 and p_2 be the paths from u to w along p and w to v along p respectively.

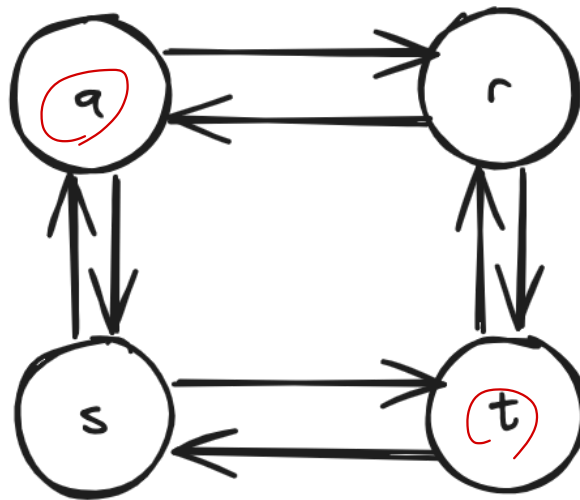


Question 74. Describe the two subproblems that p_1 and p_2 are solutions to. Is it possible that there is a better solution to these problems?

p_1 is $SP_{u,w}$ and p_2 is $SP_{w,v}$

Now let's look at the LONGESTSIMPLEPATH problem. We want to check if this problem has optimal substructure. To reiterate, this means that the solution to full problem can be written as a composition of the solutions to subproblems.

Question 75. Consider an instance of the LONGESTSIMPLEPATH problem with the following graph as input with labels q and t . What is the solution?



$LSP_{q,t} : q \rightarrow r \rightarrow t$

Question 76. Show by counterexample that LONGESTSIMPLEPATH does not have the optimal substructure property. In other words, the optimal solution cannot be described by solutions to subproblems.

$LSP_{q,r} : q \rightarrow s \rightarrow t \rightarrow r$

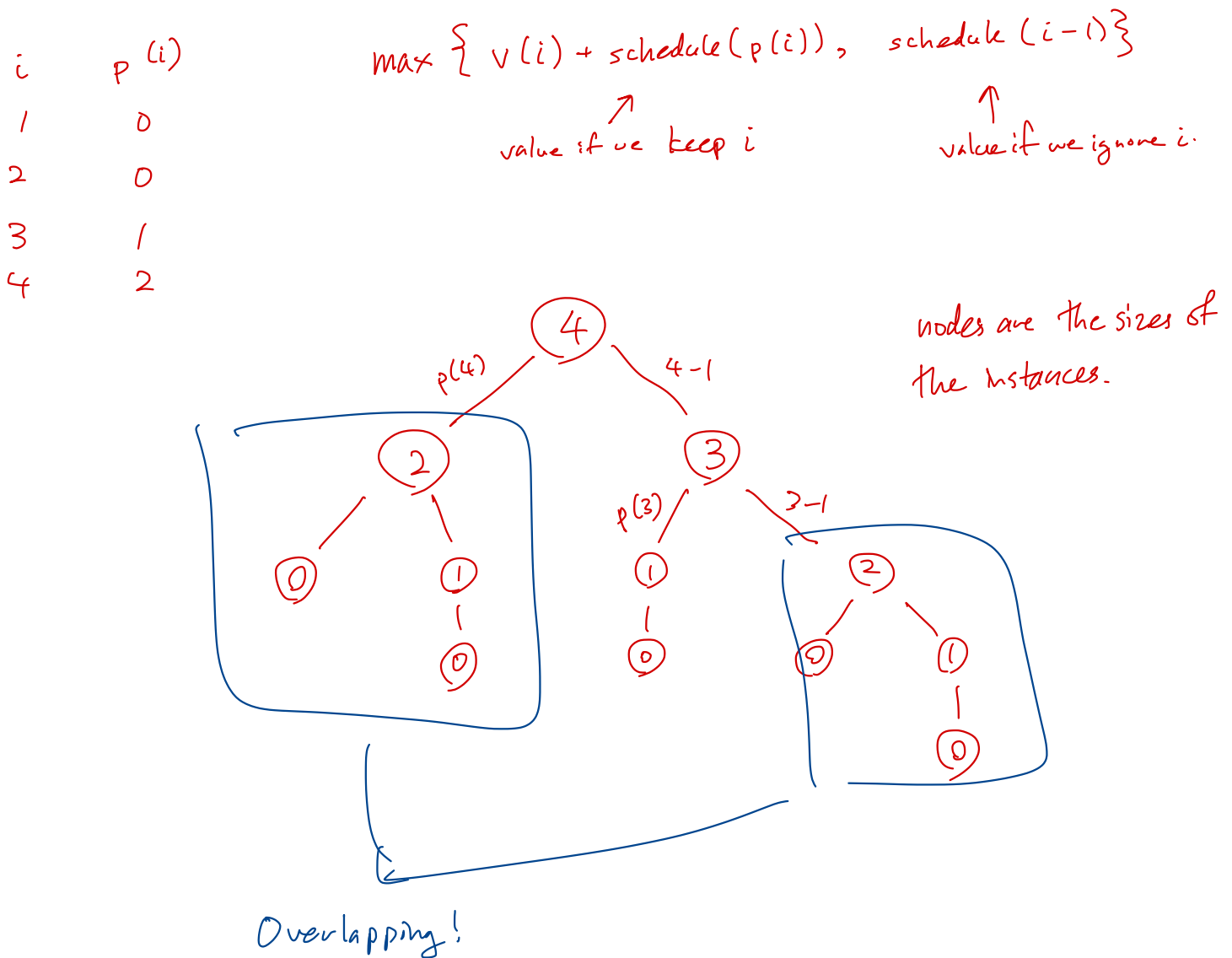
+

$LSP_{r,t} : r \rightarrow q \rightarrow s \rightarrow t$

6.3.2 Overlapping Subproblems

The second property ingredient we need in an optimization problem to apply dynamic programming is having **overlapping subproblems**. When we think of breaking our problem in to subproblems, sometimes the same subproblem is solved many different times. If this is only repeated say one or two times, it doesn't cause an issue for us. However, the number of calls can easily blow up. Recall our tree that we drew in Question 69 that showed the number of overlapping calls we made to solve the rod cutting problem.

Question 77. Does the weighted scheduling problem have the overlapping subproblem property? If so, write down a scenario where the same subproblem gets called twice.



There are two main ways that people deal with overlapping subproblems. We saw these in the rod cutting example, the top-down approach (memoization) and the bottom-up approach (tabulation). In both approaches, we are keeping track of solutions to subproblems that we have solved already so that we don't waste computation time to resolve them.

Top-Down (Memoization): To implement an algorithm using memoization, first write a recursive algorithm to solve your problem. The algorithm should be clear about what the subproblems are and how we combine the solutions to the subproblems.

Question 78. Write down a recursive algorithm to solve the weighted scheduling problem.

```
def schedule(n):
    if n == 0:
        return 0
    max_val = max(v[n] + schedule(p[n]),
                  schedule(n-1))
    return max_val
```

} Base case.

} recursive calls.

Now to create a memoized version of the algorithm, we will use an array initialized as a global variable to store the solutions to our subproblems. In this step, it is important to know how many subproblems we will have to deal with.

Question 79. For the weighted scheduling problem, how many possible subproblems are there? Remember, this problem asks "for all events 1 through i , what is the maximum value we can achieve?" for an input of i .

n different subproblems.

1..1
1..2
1..3
:
1..n

Once the memo is defined, we split our function into two parts.

1. IF this subproblem has been solved: return the solution stored in the memo.
2. ELSE: solve the problem just like we did in the recursive version.

Question 80. Write down an algorithm to solve the weighted scheduling problem using memoization.

memo : 0 , $-\infty$, $-\infty$, ... , $-\infty$
 0 1 2 ... n
 —

```
def schedule(n):
    if memo[n] ≥ 0:
        return memo[n]

    max_val = ...
    memo[n] = max_val
    return max_val
```

Bottom-Up (Tabulation): In the bottom up approach, instead of using a recursive algorithm, we will instead iterate through our table from the bottom and build up the solutions using the cells we have filled out so far. This was the way we solved the weighted scheduling problem in the first place.

In practice, the bottom-up method outperforms the top-down approach especially when you have a problem where every subproblem has to be solved at least once. First of all, function calls can be expensive depending on your system, so you can accumulate overhead from your recursive calls. The bottom-up approach also doesn't need a large external array to store the intermediate solutions. However, if your problem doesn't require looking at every subproblem for most inputs, then the top-down approach can be faster in practice.

6.3.3 Pieces to identify

The following components are also extremely helpful when defining a dynamic programming algorithm.

1. **Subproblem domain:** The space of possible subproblems to consider.
2. **Memo table definition:** The items we will be storing in our tables. Alternatively, the name of solutions to subproblems.
3. **Goal:** The location of our solution in the table.
4. **Initial values:** Solutions to trivial subproblems to start building on.
5. **Recurrence:** A compact representation of the recursive function.

Question 81. Identify the above pieces in the weighted scheduling problem.

1. Inputs can be $i \in [0, \dots, n]$

2. $\text{memo}(i)$: The optimal value achievable w/ the first i items.

3. goal : $\text{memo}(n)$

4. $\text{memo}(0)$: 0 (schedule w/ 0 events)

→ 5. $\max \{ v(n) + \text{schedule}(p(n)), \text{schedule}(n-1) \}$.

6.4 Truck Loading Problem

Problem (TRUCKLOADING).

INPUT: A truck that can hold boxes up to weight W , and a list $\{w_1, w_2, \dots, w_n\}$ of weights of n boxes to load.

OUTPUT: Which boxes to load to achieve the maximum possible weight without exceeding W .

Question 82. Write down an example instance of the truck loading problem. That is, write down a viable input output pair.

$$W = 120 \quad n = 3, \quad \{90, 40, 70\}$$

① ② ③

$$\textcircled{1} + \textcircled{2} > 120 \quad \text{X}$$

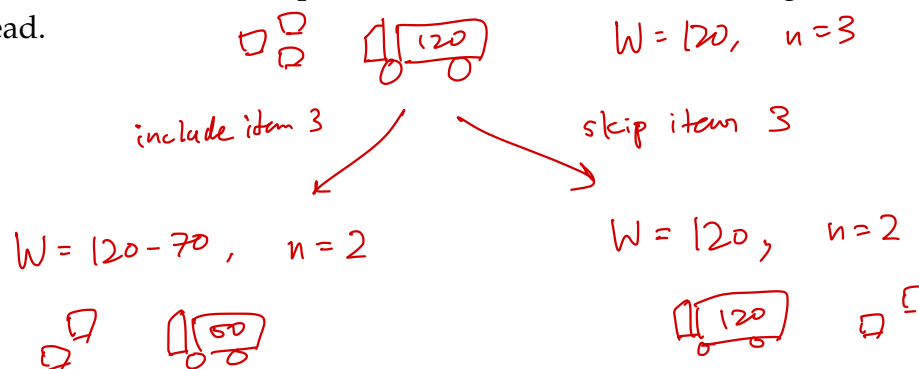
$$\textcircled{2} < 120 \quad \text{O}$$

$$\textcircled{2} + \textcircled{3} < 120 \quad \text{O} \quad \checkmark$$

↳ output = $\textcircled{110}$

In the weighted scheduling problem and rod cutting, there was one way to generate subproblems from the main problem. There are two ways to do so here, because there are two main parameters to the problem. One way to create subproblems is like the weighted scheduling problem, where we consider the first i items instead of all n items. The other way is to reduce the capacity of the truck to something smaller and see what happens.

It's important to consider both, because we can think of choosing to add the last item to a truck to induce a new subproblem which uses the remaining items, and a smaller truck instead.



Question 83. Reason about whether or not this problem has the optimal substructure property.

- What are a set of easy top level choices to make, which break the problem into multiple subproblems? Remember, a subproblem must also be a valid instance of the main problem!

① Problem statement: $TL(W, n) \rightarrow$ The max weight loadable onto a truck w/ capacity W using the first n boxes.

② Choice: $\underbrace{\text{Keep box } n}_A$ or $\underbrace{\text{skip box } n}_B$.

③ Induced subproblems.

$$A: TL(W - w[n], n - 1)$$

$$B: TL(W, n - 1)$$

- • If someone tells you the solution to this first choice, does the correct answer to the subproblems build into the correct solution of the full problem?

Yes, we can directly solve the induced subproblem after the choice, and combine that solution with our choice.

Question 84. Using the above analysis, can you define a recursive algorithm that solves the problem? This algorithm should **not** use a memo and will likely be inefficient.

```
def load_truck(W, n):
```

```
    if W <= 0 or n == 0:
        return 0
```

```
    return max(w[n] + load_truck(W - w[n], n - 1),
               load_truck(W, n - 1))
```

Inputs: W, n
 $\uparrow$ total capacity $\uparrow$ # of boxes.

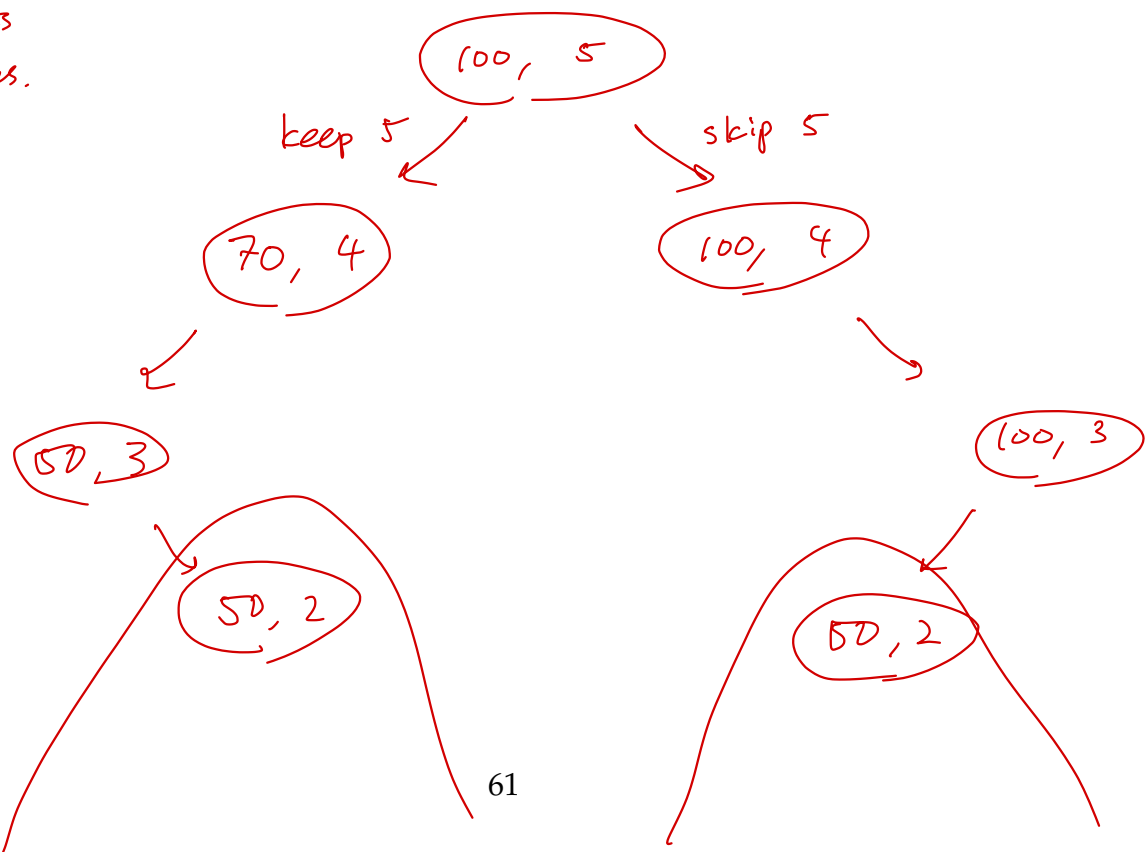
Base case:
 • Truck has no capacity, OPT. is 0.
 • No more boxes, OPT is 0.

Recursive step:

Question 85. Given your answer to the previous problem, does the truck loading problem also have overlapping subproblems? If yes, write down a scenario where the same subproblem gets called twice.

$W = 100$ $n = 5$ $\{10, 80, 50, 20, \underline{30}\}$

nodes: inputs
 edges: choices.



Question 86. Identify the pieces of the dynamic programming algorithm.

- ① **Subproblem domain:** The space of possible subproblems to consider.
- ② **Memo table definition:** The items we will be storing in our tables. Alternatively, the name of solutions to subproblems.
- ③ **Goal:** The location of our solution in the table.
- ④ **Initial values:** Solutions to trivial subproblems to start building on.
- ⑤ **Recurrence:** A compact representation of the recursive function.

① • Truck capacity could be in $\{0, \dots, W\}$
 • First j items where j in $\{0, \dots, n\}$
 $\Rightarrow (W+1)$ by $(n+1)$ subproblem domain.
 $\Rightarrow$ DP table must have this shape.

② $\text{memo}(i, j)$: The maximum weight achievable using the first j items
 in a truck w/ capacity i .

③ goal = $\text{memo}(W, n)$

④ Initial Values : $\text{memo}(0, j) = 0$
 $\text{memo}(i, 0) = 0$

⑤ Recursion : $\text{memo}(W, n) = \begin{cases} \max(w[n] + \text{TL}(W - w[n], n-1), \text{TL}(W, n-1)) & \Rightarrow \text{if } w[n] \leq W \\ \text{TL}(W, n-1) & \Rightarrow \text{if } w[n] > W \end{cases}$

The following is an example of a table you might have at the end of a dynamic programming algorithm to solve TRUCKLOADING, with an instance where $n = 3$ with $w_1 = 6$, $w_2 = 2$, $w_3 = 5$ and $W = 7$.

Remember, each cell $\text{memo}[i, j]$ stores what the optimal solution is to an instance where we use the first i items, with a truck that can have up to weight j .

Bottom Up.

$\text{memo}(1, 1)$

$\rightarrow \text{TL}(1, 0)$

$\rightarrow \text{TL}(1, 0)$

$\text{memo}(6, 3)$

$w_3 + \text{TL}(6 - w_3, 2)$
 $5 + \text{TL}(1, 2)$
 0

(5)

$\text{TL}(6, 2)$
 (6)
 1

w

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	6	
2	0	0				2	6	
3	0				2	5	6	

target
 $\text{memo}(W, n)$

$\text{memo}(6, 1)$

$\rightarrow \text{max}$
 $w_1 + \text{TL}(6 - w_1, 0)$
 $6 + 0$
 (6)
 $\text{TL}(6, 0)$
 0

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	0			0	0	0
2	0		0					0
3	0							0

Top Down

Start from $\text{memo}(7, 3)$.
 Not filled, so recurse.

$w_3 + \text{TL}(2, 2)$

$\text{TL}(7, 2)$

Ignored a lot of the table,
 so in practice Top Down may be faster.

Question 87. What is the runtime of the dynamic programming algorithm?

Must fill table w/ size $W \cdot n$

$\Rightarrow O(Wn)$

6.5 Matrix Chain Multiplication

Question 88. Suppose we have two matrices A with size p by q and B with q by r . How many multiplications do we need to compute their product AB ?

Question 89. Suppose we have three matrices such that the dimensions are

$$A : p \times q$$

$$B : q \times r$$

$$C : r \times s.$$

What is the total number of multiplications we need if we compute (AB) first? What if we compute (BC) first?

Problem (MATRIXCHAIN).

INPUT: A sequence of matrices $M_1, M_2, \dots, M_n$ and a list of dimensions $d_0, d_1, d_2, \dots, d_n$ such that M_i is size d_{i-1} by d_i .

OUTPUT: What is the most efficient way to group the matrices to minimize the total number of multiplications?

Question 90. Reason about whether or not this problem has the optimal substructure property.

- What are a set of easy top level choices to make, which break the problem into multiple subproblems? Remember, a subproblem must also be a valid instance of the main problem!
- If someone tells you the solution to this first choice, does the correct answer to the subproblems build into the correct solution of the full problem?

Question 91. Using the above analysis, can you define a recursive algorithm that solves the problem? This algorithm should **not** use a memo and will likely be inefficient.

Question 92. Given your answer to the previous problem, does the MATRIXCHAIN problem also have overlapping subproblems? If yes, write down a scenario where the same subproblem gets called twice.

Question 93. Identify the pieces of the dynamic programming algorithm.

Subproblem domain: The space of possible subproblems to consider.

Memo table definition: The items we will be storing in our tables. Alternatively, the name of solutions to subproblems.

Goal: The location of our solution in the table.

Initial values: Solutions to trivial subproblems to start building on.

Recurrence: A compact representation of the recursive function.

The following is an example of a table you might have at the end of a dynamic programming algorithm to solve MATRIXCHAIN, for the following instance.

Each cell $\text{memo}[i, j]$ stores what the optimal solution is to an instance where we consider the sequence starting from matrix i and ending at matrix j .

- $A_1 : 10 \times 15$
- $A_2 : 15 \times 5$
- $A_3 : 5 \times 60$
- $A_4 : 50 \times 100$
- $A_5 : 100 \times 20$
- $A_6 : 20 \times 40$
- $A_7 : 40 \times 47$

	1	2	3	4	5	6	7	
1	0	750	3750		41750	46750		1
	-	1	2		2	2		
2		0	4500	37500	41500	47000	56925	2
		-	2	2	2	2	2	
3			0	30000	40000	44000	53400	3
			-	3	4	5	6	
4				0	120000		214000	4
				-	4		5	
5					0	80000	131600	5
					-	5	5	
6						0	37600	6
						-	6	
7							0	7
							-	

Question 94. Write down a bottom-up algorithm to solve the MATRIXCHAIN problem.

Question 95. What is the runtime of the dynamic programming algorithm?